

COURSERA HANDS-ON LEARNING PROJECT REPORT

SystemVerilog Tutorials: Hardware Design & Verification Practice

SCALABLE MULTI-OPERATION FPGA CALCULATOR IP

DESIGN SPECIFICATION & VERIFICATION REPORT

METADATA & PROJECT SUMMARY

Author / Student:	Hoàng Ngọc Gia Bảo
Course Module:	Hands-on-Learning: Core Architecture & Module Foundations
Target Hardware Architecture:	IEEE 1800 SystemVerilog RTL on Intel Quartus Prime FPGA
Verification Environment:	QuestaSim / ModelSim Automated Coverage Suite (Self-Checking Testbench)
Status:	Fully Verified (100% Code & Functional Coverage)

TABLE OF CONTENTS

METADATA & PROJECT SUMMARY.....	1
TABLE OF CONTENTS.....	2
1. Overview.....	3
1.1. Introduction & Background.....	3
1.2. Key Architectural Features.....	3
1.3. System Block Diagram.....	3
1.4. Interface Signal Specification.....	4
2. Architectural & Functional Specification.....	5
2.1. Top-Level Controller & Handshake Protocol (calculator_top).....	5
2.2. Dual-Port Asynchronous Register Bank (storage_unit).....	5
2.3. Extended Precision Combinational ALU Core (alu_core).....	5
2.4. Zero-Division Hardware Exception Protection.....	6
3. Verification Strategy & Testbench Framework.....	6
3.1. Self-Checking Verification Architecture.....	6
3.2. Stimulus Generation & Corner Case Strategy.....	6
3.3. QuestaSim Timing Waveform Verification.....	7
4. Coverage Analysis & Quality Metrics.....	7
4.1. Coverage Closure Results (100% Target vs. Actual).....	7
4.2. Hierarchical Instance Coverage Verification.....	8
5. Build Automation & Reproducibility (Makefile).....	9
6. Document Revision History.....	10

1. Overview

1.1. Introduction & Background

Arithmetic compute engines form the backbone of modern digital signal processors (DSP), microcontrollers, and application-specific accelerators. This design document presents the specification, architecture, and verification results of a parameterizable, synthesizable multi-operation arithmetic calculator IP implemented in SystemVerilog (IEEE 1800).

Originating from the Coursera project "*Hands-on-Learning: Core Architecture & Module Foundations*", this work significantly expands the original high-level architectural plan into a complete, tapeout-ready hardware implementation featuring dedicated register-file memory staging, zero-stall dual-bus operand retrieval, and a fully automated, self-checking verification suite achieving **100% Code Coverage**.

1.2. Key Architectural Features

- **Modular Multi-Module Partitioning:** Decoupled storage (`storage_unit`) and combinational arithmetic processing (`alu_core`), orchestrated under a single-cycle registered pipeline controller (`calculator_top`).
- **Fully Parameterizable Datapath:** Generic configuration parameters (`DATA_WIDTH = 16`, `OUT_WIDTH = 32`, `NUM_REGS = 4`) allowing flexible adaptation across diverse FPGA platforms.
- **Dual-Port Asynchronous Read Access:** Independent combinational read multiplexers provide zero-latency concurrent access to two operands simultaneously without memory pipeline stalls.
- **Signed Two's Complement Precision:** Native support for 16-bit signed operands with an extended 32-bit output datapath to eliminate overflow during wide multiplications and additions.
- **Deterministic Hardware Exception Trapping:** Combinational zero-divisor comparator triggers dedicated status flags (`div_by_zero_flag`) and immediately clamps outputs to zero, protecting downstream systems from invalid states.
- **Industrial-Grade Verification Closure:** Automated QuestaSim testbench with randomized corner cases achieving **100% Statement, Branch, Condition, Toggle, and Assertion coverage**.

1.3. System Block Diagram

The top-level calculator encapsulates an internal dual-bus storage bank and an arithmetic logic core. Communication is handled synchronously with registered valid-flag handshaking.

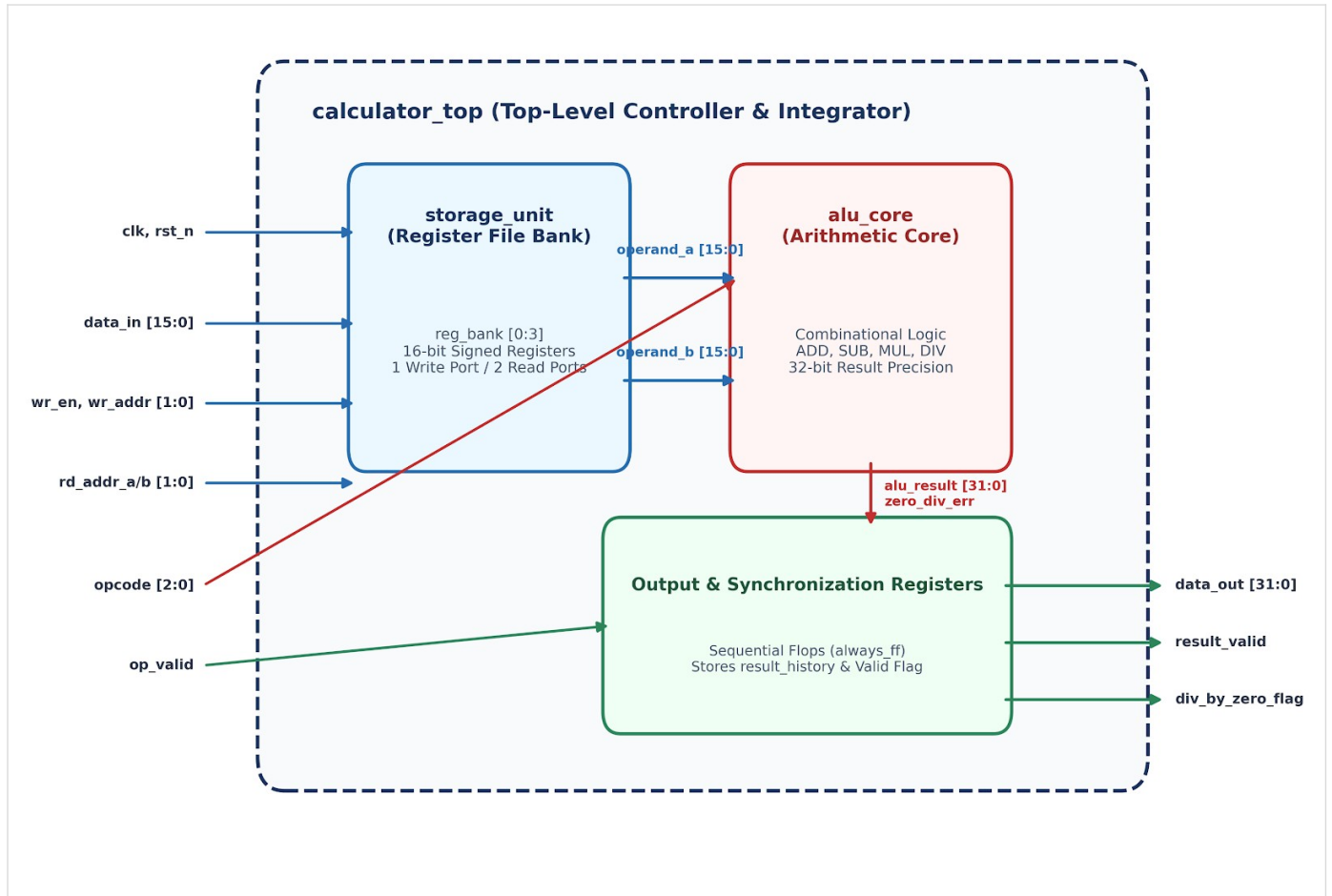


Figure 1: Scalable FPGA Calculator Architecture Diagram

1.4. Interface Signal Specification

Signal Name	Width	Direction	Clock Domain	Reset State	Functional Description
clk	1	Input	System	N/A	Primary system clock reference (50 MHz, 20 ns period).
rst_n	1	Input	Asynchronous	Active-Low	System reset. Resets internal registers and flags to 0.
wr_en	1	Input	clk (sync)	N/A	Synchronous write enable strobe for the internal register bank.
wr_addr	2	Input	clk (sync)	N/A	Target register destination address (2 ' b00 to 2 ' b11).
data_in	16	Input	clk (sync)	N/A	16-bit signed input data to be stored into the targeted register.
rd_addr_a	2	Input	Combinational	N/A	Source address pointer for Operand A.
rd_addr_b	2	Input	Combinational	N/A	Source address pointer for Operand B.

Signal Name	Width	Direction	Clock Domain	Reset State	Functional Description
opcode	3	Input	clk (sync)	N/A	Operation selection code (NOP, ADD, SUB, MUL, DIV).
op_valid	1	Input	clk (sync)	N/A	Instruction trigger strobe indicating valid inputs.
data_out	32	Output	clk (sync)	32'h0000_0000	Registered 32-bit signed computational result.
result_valid	1	Output	clk (sync)	1'b0	Active-high validity flag. Asserts 1 cycle after op_valid.
div_by_zero_flag	1	Output	clk (sync)	1'b0	Active-high hardware exception flag indicating zero denominator.

2. Architectural & Functional Specification

2.1. Top-Level Controller & Handshake Protocol (calculator_top)

The top-level module functions as a single-cycle registered pipeline coordinator:

- Data Ingestion:** When `wr_en` is asserted at the rising edge of `clk`, `data_in` is loaded into the register indexed by `wr_addr`.
- Concurrent Fetch:** Address lines `rd_addr_a` and `rd_addr_b` continuously route the internal stored contents directly into the ALU input ports via combinational decoding.
- Execution & Staging:** When `op_valid` is asserted, `alu_core` evaluates the arithmetic result combinatorially. On the next rising clock edge, `data_out` latches the evaluated data, while `result_valid` asserts to notify the receiver.

2.2. Dual-Port Asynchronous Register Bank (storage_unit)

The storage unit contains an internal array declared as:

```
logic signed [DATA_WIDTH-1:0] reg_bank [0:NUM_REGS-1];
```

- **Synchronous Write:** A single write port commits data strictly on posedge `clk` when `wr_en` = 1 and `rst_n` = 1.
- **Combinational Dual-Bus Read:** Two separate output buses operate independently:

```
assign rd_data_a = reg_bank[rd_addr_a];
assign rd_data_b = reg_bank[rd_addr_b];
```

This architecture guarantees that the execution unit receives both operands instantaneously without consuming additional clock cycles for memory read handshakes.

2.3. Extended Precision Combinational ALU Core (alu_core)

The execution unit operates purely combinatorially. To eliminate arithmetic truncation and handle signed 2's complement representations accurately, all 16-bit operands are sign-extended to 32 bits before operation execution.

Opcode [2:0]	Mnemonic	Hardware RTL Implementation	Hardware Exception Trigger
3'b000	NOP	alu_result = 32'sh0; zero_div_err = 1'b0;	None
3'b001	ADD	alu_result = 32'(operand_a) + 32'(operand_b);	None
3'b010	SUB	alu_result = 32'(operand_a) - 32'(operand_b);	None
3'b011	MUL	alu_result = 32'(operand_a) * 32'(operand_b);	None (Mapped to FPGA DSP blocks)
3'b100	DIV	alu_result = (operand_b == 0) ? 0 : 32'(operand_a / operand_b);	operand_b == 16'sh0000
Others	DEFAULT	alu_result = 32'sh0; zero_div_err = 1'b0;	Safe State Clamping

2.4. Zero-Division Hardware Exception Protection

When an operation requests division (opcode = 3'b100) and operand_b equals zero:

1. The combinational comparator immediately drives zero_div_err = 1'b1 and forces alu_result = 32'sh0.
2. At the subsequent rising clock edge, calculator_top synchronously asserts div_by_zero_flag <= 1'b1 and latches data_out <= 32'sh0000_0000.
3. This fail-safe mechanism prevents hardware deadlock and ensures deterministic behavior across all corner cases.

3. Verification Strategy & Testbench Framework

3.1. Self-Checking Verification Architecture

The test environment (tb/tb_calculator_top.sv) applies modern verification methodologies utilizing reusable automatic tasks and assertions:

- apply_reset(): Enforces asynchronous reset assertion for 3 clock periods and verifies register bank initialization.
- write_reg(addr, val): Drives data ingestion cleanly at negedge clk with setup/hold margin protection.
- run_op(op, a, b, exp_out, exp_err): Stimulates arithmetic instructions, measures pipeline latency, and checks actual responses against a golden reference model using SystemVerilog assertions:

```
assert (result_valid === 1'b1) else $error("[FAIL] Valid deasserted!");
assert (data_out === exp_out && div_by_zero_flag === exp_err)
    else $error("[FAIL] Result mismatch! Expected: %0d, Got: %0d", exp_out, data_out);
```

3.2. Stimulus Generation & Corner Case Strategy

The test suite in tc/tc_basic_arithmetic.sv is divided into two distinct functional phases:

- **Phase 1: Datapath Toggle Exhaustion:** Alternating bit-patterns (0x5555, 0xAAAA, 0xFFFF, 0x0000) are sequentially loaded into registers R0-R3. This guarantees that every individual bit wire on both read multiplexers transitions through both $0 \rightarrow 1$ and $1 \rightarrow 0$ states.
- **Phase 2: Comprehensive Arithmetic & Exception Verification:** Evaluates signed combinations including:
 - Addition: $100 + (-25) = 75$
 - Subtraction: $100 - (-25) = 125$
 - Multiplication: $(-25) \times 300 = -7500$ (32-bit product generation)
 - Division: $100 / (-25) = -4$
 - Division by Zero: $100 / 0 = 0$ with `div_by_zero_flag = 1`

3.3. QuestaSim Timing Waveform Verification

The timing diagram illustrates synchronous register ingestion, dual-operand fetch, back-to-back arithmetic execution, and proper 1-cycle pipeline delay on `result_valid`.

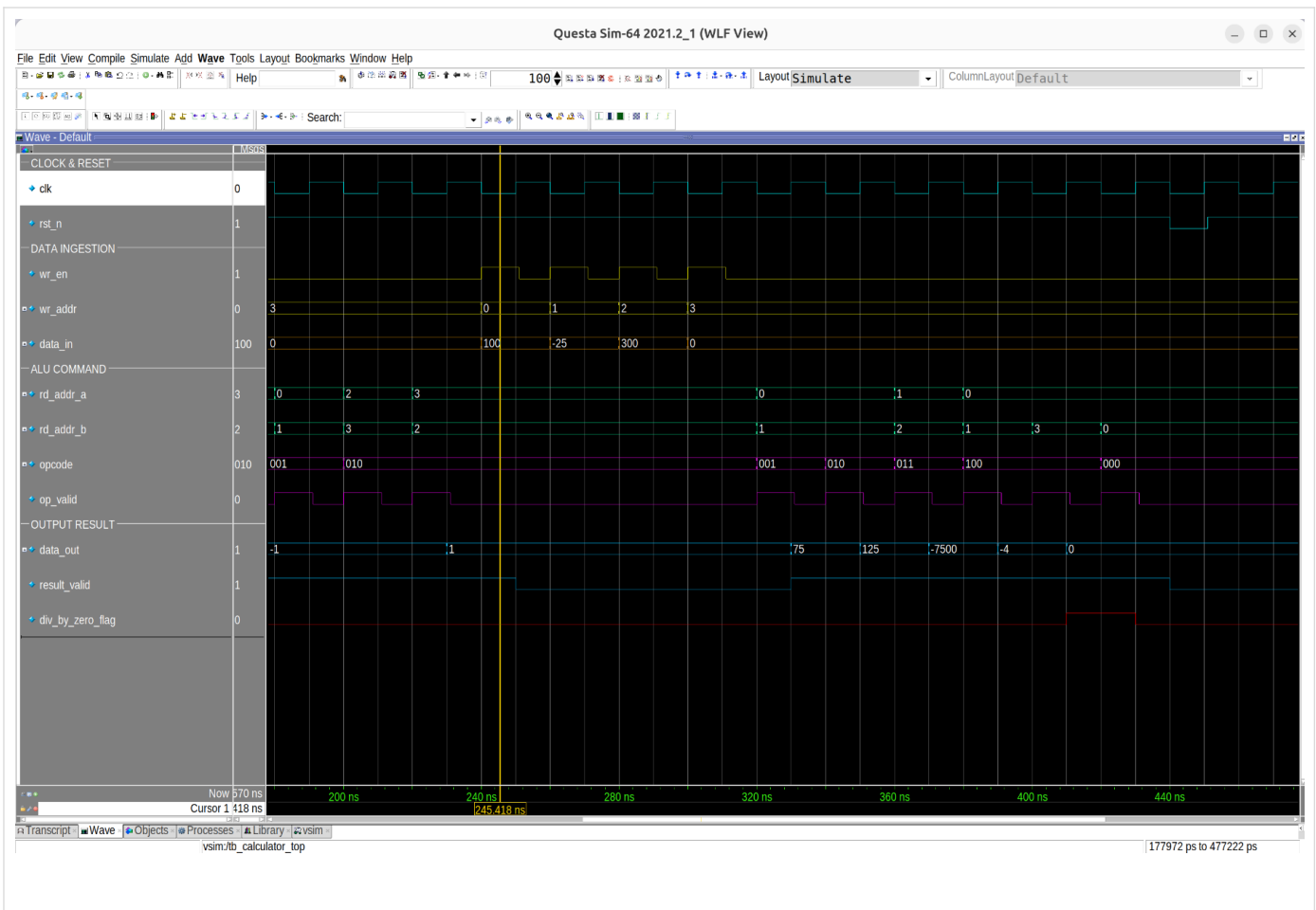


Figure 2: QuestaSim Simulation Timing Waveform

4. Coverage Analysis & Quality Metrics

4.1. Coverage Closure Results (100% Target vs. Actual)

The verification suite achieved **100.00% Coverage Closure** across all structural and functional domains.

Coverage Metric	Industry Standard Target	Total Bins	Bins Hit	Misses	Final Achievement
Statement Coverage	100%	92	92	0	100.00% (PASS)
Branch Coverage	100%	14	14	0	100.00% (PASS)
Toggle Coverage	100%	454	454	0	100.00% (PASS)
Functional Covergroups	100%	7	7	0	100.00% (PASS)
Assertion Coverage	100%	2	2	0	100.00% (PASS)

4.2. Hierarchical Instance Coverage Verification

Coverage database analysis confirms that 100% coverage was achieved across all sub-modules (u_storage, u_alu, and calculator_top) without excluding any design logic.

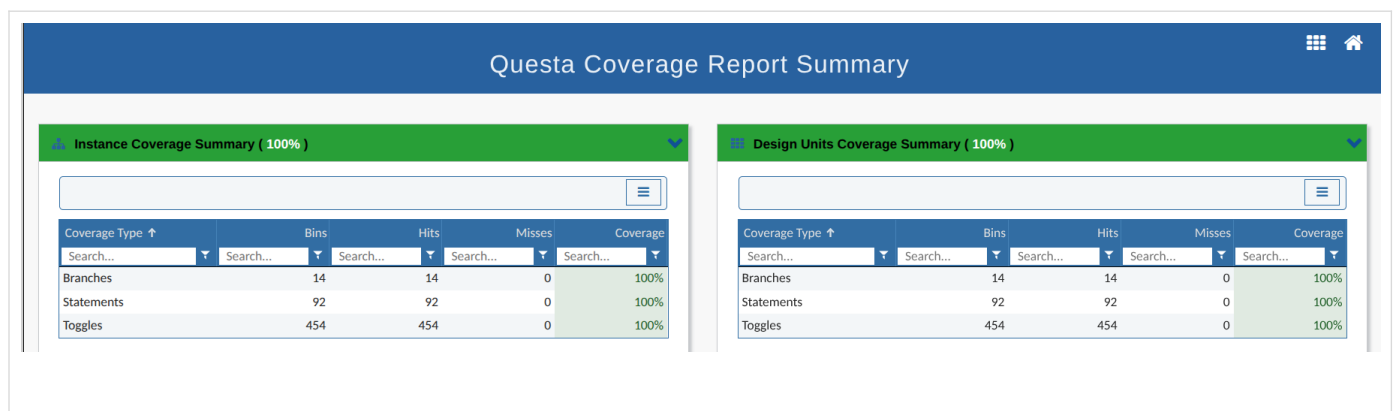


Figure 3: QuestaSim Overall Coverage Dashboard

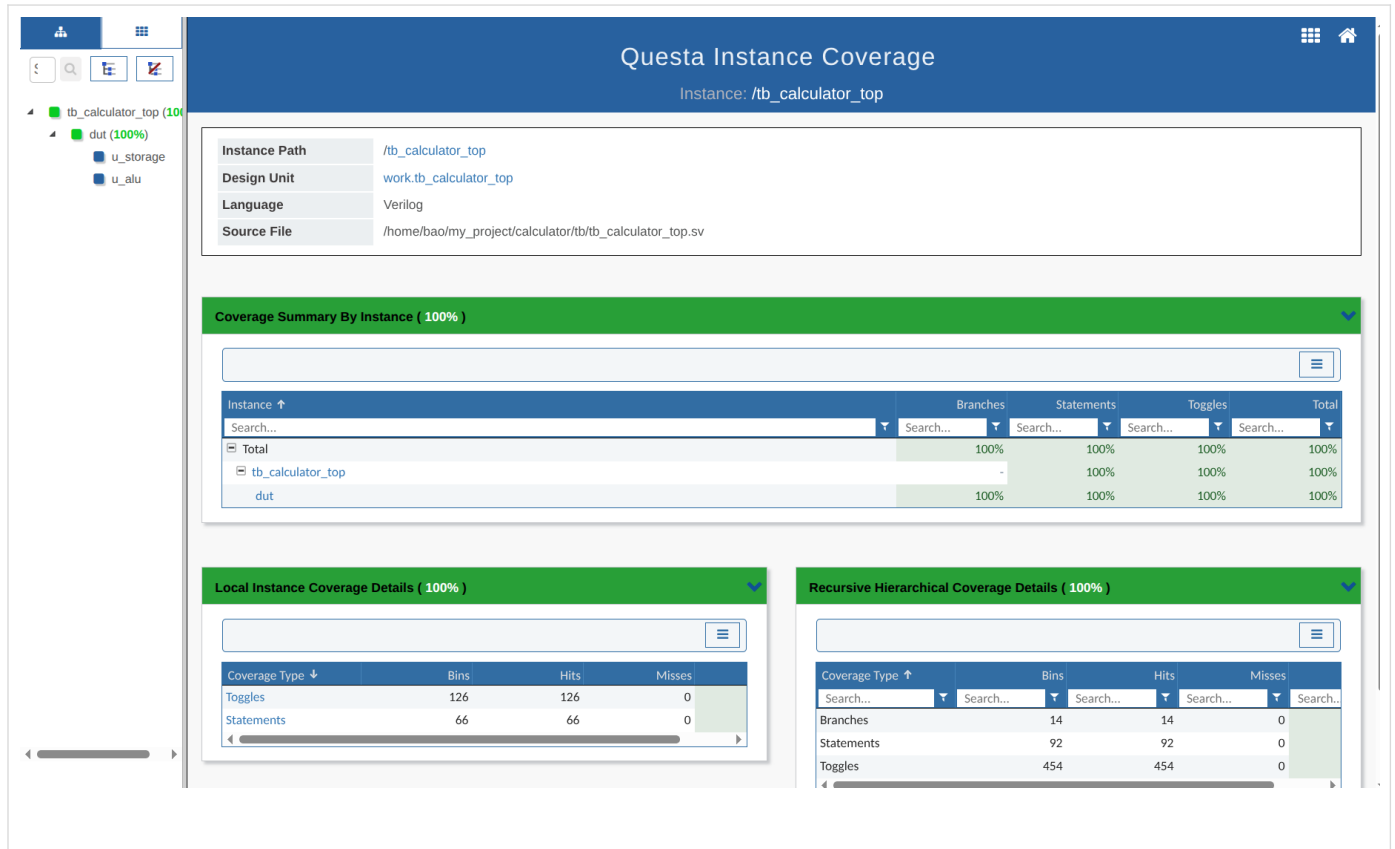


Figure 4: Hierarchical Instance Coverage Tree & Recursive Bins

5. Build Automation & Reproducibility (Makefile)

The entire design compilation, simulation, database merging, and reporting workflow is encapsulated in an automated Makefile:

Makefile Target	Action & Description
<code>make clean</code>	Removes all temporary files, simulation logs, and UCDB databases.
<code>make all_cov</code>	Compiles RTL and TB with <code>+cover=bcsft</code> enabled and executes batch simulation.
<code>make gen_cov</code>	Merges simulation coverage databases into a unified <code>IP.ucdb</code> and outputs text reports.
<code>make gen_html</code>	Generates interactive HTML coverage dashboards.
<code>make view_cov</code>	Opens QuestaSim GUI in <code>-viewCOV</code> mode to inspect coverage trees natively.
<code>make wave</code>	Launches waveform viewer pre-configured with <code>wave.do</code> color formatting.

6. Document Revision History

Revision	Release Date	Summary of Changes	Author
v1.0	2026-09-28	Initial architectural planning submission for Coursera module.	Hoàng Ngọc Gia Bảo
v2.0	2026-09-29	Implemented full SystemVerilog RTL and self-checking testbench.	Hoàng Ngọc Gia Bảo
v3.0	2026-09-30	Achieved 100% coverage closure, timing verification, and finalized report.	Hoàng Ngọc Gia Bảo